

— قصة ملف واحد — سطراً سطراً

PacketRelayManager.kt

هذا ملف bitchat story : أبو حيدر
يفتح ملفاً حقيقياً واحداً من تطبيق
bitchat-android — اسمه

PacketRelayManager.kt (قلب «وحش

العاصفة»: مين يعيد بث الرسالة ومين
لا). ويشرح لسُكينة كل سطر فيه،
من الأول للأخير، بدون نسيان أي
سطر. الملف الأصلي ٢١٧ سطراً،
وراح نمرّ عليها كلها.

كل مقطع: نوريك الأسطر الحقيقية
بأرقامها، وبعدين أبو حيدر يقول «هنا
فعلنا كذا، وأضفنا هذي الأسطر
عشان...».

المقدّمة: نفتح الملف

جلس أبو حيدر، وسُكينة جنبه، وفتح ملفاً على الشاشة.

«يا سُكينة، اليوم ما راح أحكي لك فكرة عامة. اليوم راح نقرأ ملفاً حقيقياً واحداً، سطرًا سطرًا، مثل ما نقرأ صفحة من كتاب. اسمه `PacketRelayManager.kt`. شغلته وحدة: لمن توصل رزمة مو إلنا، يقرّر: نعيدها للجيران أو لا؟ هذا قلب وحش العاصفة.»

«كل سطر يا جدّو؟ حتى الفاضية؟»

«حتى الفاضية يا حبيبي! كل سطر له سبب — حتى الفراغ، نحطه عشان

«يتنفس» الكود ونقدر نقراه. يلا نبدأ من
السطر الأول.»

الأسطر ١-٩: من نحن وماذا نحتاج؟ (العنوان والاستيرادات)

```
1 package com.bitchat.android.mesh  
  
2 import  
com.bitchat.android.protocol.MessageType  
  
3  
  
4 import android.util.Log  
  
5 import  
com.bitchat.android.model.RoutedPacket  
  
6 import  
com.bitchat.android.protocol.BitchatPacket  
  
7 import  
com.bitchat.android.util.toHexString
```

```
8 import kotlinx.coroutines.*  
9 import kotlin.random.Random
```

«شوفي يا سُكينة، السطر ١ يقول:

— `package com.bitchat.android.mesh`

يعني هذا الملف يسكن بـ"حي" اسمه

`mesh` (الشبكة). مثل عنوان البيت: أي

مهندس يعرف وين يلقاه.»

«والباقي يا جدّو؟»

«الأسطر ٢ و٤ لين ٩ كلها `import`

— يعني "استورد". مثل لمن تطبخين

وتجيبين الأدوات قبل ما تبدئين: ملعقة،
صحن، سكين. هنا نجيب أدواتنا:

• السطر ٤: `Log` — أداة نكتب فيها

ملاحظات عشان نعرف شنو صار
(مثل دفتر يوميات للكود).

• السطر ٥: `RoutedPacket` — الرزمة

مع معلومات طريقها.

• السطر ٦: `BitchatPacket` — الرزمة

نفسها.

• السطر ٧: `toHexString` — أداة

تحوّل الأرقام لنصّ ست عشري
(شكل خاص للأسماء).

• السطر ٨: `kotlinx.coroutines.*` —

أدوات تخليّننا نشغل بدون ما نوقف
التطبيق (تذكرين `suspend` ؟ هذي

أدواتها). والنجمة * معناها "جيب

كل الأدوات من هالصندوق".

• السطر ٩: Random — أداة

العشوائية! بنحتاجها عشان "نرمي زهر"

ونقرر نعيد البث أو لا. تذكرين

حكمة العاصفة؟»

«والسطر ٣ الفاضي؟»

«السطر ٣ فراغ، عشان نفصل أداة عن

أداة، فيرتاح النظر. الفراغ مو نسيان —

هو ترتيب.»

الأسطر ١١-١٦: لوحة التعريف (شنو يسوّي هالملف؟)

11 /**

12 * Centralized packet relay management

13 *

14 * This class handles all relay
decisions and logic for bitchat packets.

15 * All packets that aren't specifically
addressed to us get processed here.

16 */

«الأسطر ١١ لين ١٦ هذي لوحة تعريف
(تعليق). الرموز `/**` و `*/` تقول للكود:
"هذا كلام للإنسان، مو أوامر — تجاهله
وأنت تشتغل." بس للإنسان اللي يقرأ،
تشرح:

• السطر ١٢: "إدارة تمرير الرزم، مركزية"
— يعني كل قرارات الإعادة بمكان
واحد.

• السطر ١٤: "هالكلاس يتعامل مع كل

قرارات ومنطق التمرير."

• السطر ١٥: "كل رزمة مو موجّهة

إلنا تتعالج هنا."

شفتِ يا سُكينة؟ قبل ما نكتب أي

أمر، نكتب لماذا الملف موجود. مثل

عنوان القصة قبل ما تبدأ. وهذا أدب

المهندس الزين: يشرح نفسه.»

«يعني أول ما تفتح الملف، تعرف شغلته

فوراً!»

«بالضبط! اللوحة توفّر عليك ساعة بحث.»

السطر ١٧: ولادة المدير

```
17 class PacketRelayManager(private val  
myPeerID: String) {
```

«السطر ١٧ — هنا يولد المدير! class
PacketRelayManager يعني "أنشئ مديراً اسمه
مدير-تمرير-الرزم". وبين القوسين (private val
myPeerID: String) — هذا أهم شيء:
المدير لمن يولد، لازم نعطيه هويته =
myPeerID (معرفي أنا).»

«ليش يحتاج يعرف هويته يا جدّو؟»
«سؤال ممتاز! لأنه لازم يميّز: "هذي الرزمة
مني أنا؟ ولا إلي أنا؟" لو ما يعرف
مين هو، ما يقدر يقرّر. و val تذكرين
معناها ثابت — هويته ما نتغيّر.

و `private` يعني "سرّي"، بس داخل
هالمدير" — ما أحد بره يلمسها. و `String`
يعني الهوية نصّ (حروف وأرقام).»
«ولادة مدير بسطر واحد!»

«إي، والقوس { } بآخر السطر يفتح بيت
المدير — كل اللي جاي جواه شغله.
وبآخر الملف (السطر ١٩٢) يصير قوس
} يسكّر البيت.»

السطر ١٨: مساعد التنقيح (debugManager)

```
18 private val debugManager by lazy
{ try
{ com.bitchat.android.ui.debug.DebugSettings
```

```
Manager.getInstance() } catch (e: Exception)
{ null } }
```

«السطر ١٨ — هذا مساعد صغير اسمه
debugManager، شغلته يساعد المهندسين
يراقبون السلوك أثناء التجربة. وفيه ثلاث
كلمات حلوة:

• by lazy = "كسول" — يعني ما
تجهّز بالمساعد إلا أول ما نحتاجه
فعلاً. مثل ما ما تشغلّين الفرن إلا
لمن تبين تخبزين. توفير!

• try { ... } catch (e: Exception)
{ null } = "حاول... وإذا فشلت،
لا تنهار، بس رجّع لا شيء
(null)". مثل لمن تحاولين تفتحين

علبة، وإذا ما انفتحت، ما تزعلين،
تكلّين شغلّك.»

«يعني يحاول، وإذا ما قدر، يكلّ بهدوء؟»

«بالضبط! هذا مهم بالشبكة — ما نبي
شي صغير يطّيح كل شي. نحاول، ونكلّ.»

الأسطر ٢٠-٢٢: الاسم على الدفتر (companion object)

```
20      companion object {  
21          private const val TAG =  
"PacketRelayManager"  
22      }
```

«الأسطر ٢٠ لين ٢٢ — تذكّرين دفتر
اليوميّات (Log)؟ لمن نكتب فيه، نبي

نعرف من اللي كتب. فنحط توقعاً ثابتاً

اسمه TAG (وسم) قيمته

«"PacketRelayManager"»

«لش const و val سوا؟»

«const val يعني ثابت ثابت — معروف

من قبل التشغيل وما يتغير أبداً. والاسم

كبير TAG لأنه ثابت (تذكرين قاعدة

الثابت بأحرف كبيرة؟)»

و companion object يعني "صندوق مشترك"

— التوقيع واحد لكل المدير. فأني

ملاحظة بالدقتر تجي موقعة:

"PacketRelayManager": صار كذا". فلو فيه

مشكلة، نعرف من أي ملف جت.»

«مثل ما أوقع رسيتي باسمي!»

«تماماً يا فنّانة.»

الأسطر ٢٤-٢٦: زرّ التشغيل/الإطفاء (isRelayEnabled)

```
24     private fun isRelayEnabled():  
Boolean = try {  
25  
com.bitchat.android.ui.debug.DebugSettingsMa  
nager.getInstance().packetRelayEnabled.value  
26     } catch (_: Exception) { true }
```

«الأسطر ٢٤ لين ٢٦ — دالة صغيرة

اسمها `isRelayEnabled` = "هل التمرير

مفعّل؟". تذكرين قاعدة `is`؟ الدالة اللي

تبدأ بـ `is` جوابها نعم/لا (Boolean)».

«آها! سؤال نعم/لا!»

«برافو! هذي تشوف إعداد: "هل صاحب

التلفون مفعّل خاصية إعادة البث؟" —

مفيدة وقت التجربة، لو المهندس يبي

يطفّي الإعادة مؤقتاً. والسطر ٢٦: `catch`

`{ true } ( _ )` — يعني "لو صار خطأ

بقراءة الإعداد، اعتبره مفعّل (`true`)

افتراضياً". لأن الأصل إننا نساعد بإعادة

البث، فلو ما عرفنا، نساعد. والشرطة `_`

مكان اسم الخطأ معناها "ما يهمّني تفاصيل

الخطأ، بس تجاهله".»

«يعني الأصل المساعدة، إلا إذا أحد

طفّاها بنفسه؟»

«بالضبط يا حبيبي. كرماء بالأصل.»

السطر ٢٨ والأسطر ٣٠-٣١: المندوب (delegate)

```
28      // Logging moved to  
  
BluetoothPacketBroadcaster per actual  
transmission target  
  
29  
  
30      // Delegate for callbacks  
  
31      var delegate:  
  
PacketRelayManagerDelegate? = null
```

«السطر ٢٨ تعليق سطر واحد (يبدأ بـ //) يقول: "نقلنا كتابة السجلّ لمكان ثاني." — ملاحظة للمهندسين، مو أمر.

السطر ٣٠ تعليق: "المندوب للنداءات."

السطر ٣١ مهم جداً: `var delegate` — هذا المندوب! تذكرين المايسترو والمدراء؟

هذا المدير ما يعرف كل شي عن الشبكة — فعنده "مندوب" يسأله. مثلاً: "كم عدد الجيران؟" يسأل المندوب. الكلمة `var` (مو `val`) لأن المندوب يتعين لاحقاً. و؟ بعد النوع معناها "ممکن يكون موجود أو لا" (`null`) بالبداية، لين ما يتعين.»

«ليش ما يعرف بنفسه عدد الجيران؟»
«عشان كل مدير يسوي شغلته بس. مدير الإعادة يقرر الإعادة، ومدير ثاني يعدّ الجيران. فلن يحتاج العدد، يسأل المندوب. هذا اسمه "فصل المسؤوليات" — كل واحد متخصص.»

الأسطر ٣٣-٣٤: عمّال الخلفية (relayScope)

```
33      // Coroutines
34      private val relayScope =
CoroutineScope(Dispatchers.IO +
SupervisorJob())
```

«السطر ٣٤ ينشئ "ساحة عمل" اسمها `relayScope` للأعمال التي تشتغل بالخلفية بدون ما توقف التطبيق. شوفي المكونات:

- `Dispatchers.IO` = "عمّال متخصصون بالإدخال/الإخراج" (شبكة، ملفات) —
- مو على الخيط الرئيسي التي يحرك الشاشة.

• `SupervisorJob()` = "مشرف" حكيم:

لو عامل واحد فشل، الباقي يكملون

— ما يطيحون كلهم بسبب واحد.»

«مثل مدير ورشة، لو غاب عامل، الباقي

يشتغلون؟»

«بالضبط يا سُكينة! المشرف يخلي الورشة

تكمل حتى لو تعثر واحد. السطر ٣٣

تعليق بس يقول "Coroutines" (العمال غير

المتزامنين)»

الأسطر ٣٦-٤٤: الباب الرئيسي (يبدأ `handlePacketRelay`)

```
36      /**
```

```
37      * Main entry point for relay
```

```
decisions
```

```

38      * Only packets that aren't
specifically addressed to us should be
passed here
39      */
40      suspend fun
handlePacketRelay(routed: RoutedPacket) {
41          val packet = routed.packet
42          val peerID = routed.peerID ?:
"unknown"
43
44          Log.d(TAG, "Evaluating relay
for packet type ${packet.type} from $
{peerID} (TTL: ${packet.ttl})")

```

«وصلنا القلب يا سُكينة! الأسطر ٣٦ لين
٣٩ لوحة تعريف تقول: "هذا الباب

الرئيسي لقرارات الإعادة. بس الرزم اللي
مو موجهة إلنا تجي هنا."

السطر ٤٠: `suspend fun`

`handlePacketRelay(routed: RoutedPacket)`

— الدالة الكبرى. `suspend` يعني "ممکن

تاخذ وقت، لا توقف التطبيق".

و `handlePacketRelay` = "تعامل مع تمرير

الرزمة". و تاخذ `routed` = الرزمة مع

طريقها.

السطر ٤١: `val packet = routed.packet`

— نطلع الرزمة نفسها من داخل المغلف.

السطر ٤٢: `val peerID = routed.peerID ?:`

"unknown" — نطلع مين أرسلها. وشوفي

الرمز `?:` (يسمونه إلفيس، يشبه شعره!)

معناه: "لو ما فيه مرسل معروف، سمّه

unknown (مجهول)". حماية ذكية من

الفراغ. السطر ٤٤: نكتب بالدقتر: "أقيم إعادة رزمة نوعها كذا، من فلان، عداد قفزاتها كذا." عشان لو صار شي، نعرف القصة.»

«كل خطوة يكتبها بالدقتر؟»

«إي، عشان لو خربت الشبكة، نفتح الدقتر ونعرف وين المشكلة. التوثيق صديق المهندس.»

الأسطر ٤٦-٥٦: حارسان عند الباب

```
46          // Double-check this packet  
isn't addressed to us  
  
47          if  
(isPacketAddressedToMe(packet)) {
```

```

48             Log.d(TAG, "Packet
addressed to us, skipping relay")
49             return
50         }
51
52         // Skip our own packets
53         if (peerID == myPeerID) {
54             Log.d(TAG, "Packet from
ourselves, skipping relay")
55             return
56         }

```

«قبل ما نقررّ الإعادة، فيه حارسان يا
سُكينة:

الحارس الأول (الأسطر ٤٦-٥٠): `if`
`(isPacketAddressedToMe(packet))` — "هل
الرزمة إلي أنا؟" لو إي، نكتب بالدفتر

"موجهة إلنا، ما نعيدها"، و `return` =
"ارجع، خلاص، طلعة". ليش ما نعيدها؟
لأنها وصلت هدفها (أنا)! ما نحتاج نمررها
لغيري.

الحارس الثاني (الأسطر ٥٢-٥٦): `if`
`(peerID == myPeerID)` — "هل أنا اللي
أرسلتها أصلاً؟" لو إي، `return` — ما
أعيد رزمتي لنفسي! يصير دوران سخيف.
«مثل لمن ما أعيد الرسالة لنفسي!»

«بالضبط! الحارسان يمنعان شغلاً بلا فائدة:
ما نعيد اللي إلنا، وما نعيد اللي منا.
والسطر ٥١ فاضي عشان يفصل الحارسين.»

الأسطر ٥٨-٦٦: بطارية الرسالة (TTL)

```
58          // Check TTL and decrement
59          if (packet.ttl == 0u.toUByte())
60          {
61              Log.d(TAG, "TTL expired,
not relaying packet")
62              return
63          }
64          // Decrement TTL by 1
65          val relayPacket =
packet.copy(ttl = (packet.ttl -
1u).toUByte())
66          Log.d(TAG, "Decrementd TTL
from ${packet.ttl} to ${relayPacket.ttl}")
```

«الحين تذكرين عدّاد القفزات (TTL)؟ مثل بطارية للرسالة!

الأسطر ٥٨-٦٢:

```
— "هل العدّاد if (packet.ttl == 0u...)
```

صفر؟" لو إي، نكتب "انتهى العدّاد، ما نعيدها"، و `return`. لأن الرسالة قطعت قفزاتها المسموحة، خلاص نتوقّف. الرمز `0u` يعني صفر موجب (بدون سالب)، و `toUByte()` يحوّله للنوع الصح.

السطر ٦٥: أهم سطر بالعدّاد!

```
packet.copy(ttl = (packet.ttl - 1u)...) —
```

نأخذ نسخة من الرزمة، وننقص عدّادها واحد. شوفي `copy` — ما نعدّل الأصل، نسوي نسخة جديدة اسمها

relayPacket (رزمة الإعادة) بعدّاد أنقص.

ليش نسخة؟»

فكرت سُكينة... «عشان... ما نخرب

الأصل؟»

«ذكية! نخلي الأصل سالماً ونشتغل على

نسخة. هذا أأمن. السطر ٦٦ يكتب

بالدفتر: "نقصت العدّاد من كذا لكذا".

والسطر ٦٣ فاضي، والسطر ٦٤ تعليق

يقول "أنقص واحد".»

«كل قفزة تاكل من بطارية الرسالة!»

«وبالضبط لمن تخلص، نتوقف. حلو، صح؟»

الأسطر ٦٨-٩٨: الطريق الذكي (هل أعرف الخطوة الجاية؟)

«يا سُكينة، هذا أصعب مقطع، بس أحلاه. أحياناً الرزمة تحمل خريطة طريق — قائمة بالتلفونات اللي لازم تمرّ عليها. لو الخريطة موجودة، نحاول نوصلها للخطوة الجاية بالضبط بدل ما نصيح للكل.»

```
68          // Source-based routing: if
route is set and includes us, try targeted
next-hop forwarding

69          val route = relayPacket.route

70          if (!route.isNullOrEmpty()) {

71              // Check for duplicate hops
to prevent routing loops

72              if (route.map
{ it.toHexString() }.toSet().size <
```

```

route.size) {
73             Log.w(TAG, "Packet with
duplicate hops dropped")
74             return
75         }

```

السطر ٦٩: `val route =`

— `relayPacket.route` نطلع خريطة

الطريق. السطر ٧٠: `if (!`

— `route.isNullOrEmpty()` "هل الخريطة

موجودة وموفاضية؟" (علامة `!` معناها

"ليس"؛ يعني: ليست فارغة). السطر ٧٢:

هنا كشف الدوران الذي! `route.map`

— `{ it.toHexString() }.toSet()` نحول

الخريطة لـ "مجموعة بلا تكرار"، ولو جمها

أصغر من الخريطة الأصلية، معناها فيه اسم

مكرّر = الرزمة لفت بدائرة! وقتها (السطر

(٧٣-٧٤) نكتب تحذير `Log.w` (الحرف

`w` = warning = تحذير) ونرميها

بـ `return`.

«مثل ما عرفنا إننا لفينا لو شفنا نفس

الشارع مرتين!»

«تماماً يا حبيبي، نفس الفكرة بالضبط!»

```
76          val myIdBytes =  
hexStringToPeerBytes(myPeerID)  
77          val index =  
route.indexOfFirst  
{ it.contentEquals(myIdBytes) }  
78          if (index >= 0) {  
79              val nextHopIdHex:  
String? = run {
```

```

80                val nextIndex =
index + 1

81                if (nextIndex <
route.size) {

82
route[nextIndex].toHexString()

83                } else {

84                // We are the
last intermediate; try final recipient as
next hop

85
relayPacket.recipientID?.toHexString()

86                }

87                }

```

«الحين نبحث عن أنفسنا بالخريطة: السطر
٧٦: myIdBytes — نحول هويتنا لصيغة

بايتات عشان نقارن. السطر ٧٧:

route.indexOfFirst { ... } — "وين أنا

بالخريطة؟" نلقى موقعنا (index). السطر

٧٨: if (index >= 0) — "هل أنا فعلاً

بالخريطة؟" (لو . أو أكثر، إي). الأسطر

٧٩-٨٧: نطلع الخطوة الجاية

(nextHopIdHex): - السطر ٨٠:

nextIndex = index + 1 — الخطوة بعدي.

- السطر ٨١-٨٢: لو فيه خطوة بعدي

بالخريطة، خذ اسمها. - السطر ٨٣-٨٥:

وإلا (أنا آخر واحد بالخريطة!)، جرب

أوصلها للمستلم النهائي مباشرة.

شوفي يا سُكينة: التطبيق يحاول يكون دقيقاً

— بدل ما يصيح للكل، يوصلها للجار

الصح بالضبط. أذكى وأهدأ.»

```

88             if (nextHopIdHex !=
null) {
89                 val success = try
{ delegate?.sendToPeer(nextHopIdHex,
RoutedPacket(relayPacket, peerID,
routed.relayAddress)) } catch (_: Exception)
{ false } ?: false
90                 if (success) {
91                     Log.i(TAG, "
Source-route relay: ${peerID.take(8)} -> $
{nextHopIdHex.take(8)} (type ${'$'}
{packet.type}, TTL ${'$'}
{relayPacket.ttl})")
92                     return
93                 } else {
94                     Log.w(TAG,
"Source-route next hop $

```

```

{nextHopIdHex.take(8)} not directly
connected; falling back to broadcast")
95                                     }
96                                     }
97                                     }
98                                     }

```

«السطر ٨٨: `if (nextHopIdHex != null)`

— "هل لقينا خطوة جاية؟" السطر ٨٩:

نحاول نرسلها للخطوة الجاية بالضبط عبر

المندوب `delegate?.sendToPeer(...)` ونخطّ

النتيجة بـ `success` (نجح؟). ولاحظي `try`

`{ ... } catch { false }`. — لو فشل

الإرسال، النتيجة `false` بدل ما ينهار.

السطر ٩٠-٩٢: لو نجح (`success`)،

نكتب بالدفتر "وصلتها من فلان للخطوة

الجابة"، و return — خلاص، شغلنا انتهى

بنجاح! السطر ٩٣-٩٥: لو فشل (الجار

الجاي مو متصل فينا مباشرة)، نكتب

تحذير: "ما قدرت أوصلها مباشرة، بنرجع

للطريقة العادية (الصياح للكل)".

والأسطر ٩٦-٩٧-٩٨ كلها أقواس } تسكر

الشروط اللي فتحناها. شوفي كيف كل

قوس مفتوح لازم يتسكر — مثل

الأبواب، ما نترك باباً مفتوحاً.»

«يعني يحاول الطريق الدقيق أول، وإذا ما

نفع، يصيح للكل؟»

«بالضبط يا روح جدك! يجرب الأدب

أول (التوصيل المباشر)، وإذا ما نفع،

يستعمل الصوت العالي (الصياح للكل).
والصياح هذا نشرحه بالمقطع الجاي.»

الأسطر ١٠٠-١٠٧: القرار الأخير (نعيد أو لا؟)

```
100          // Apply relay logic based on
packet type and debug switch
101          val shouldRelay =
isRelayEnabled() &&
shouldRelayPacket(relayPacket, peerID)
102          if (shouldRelay) {
103
relayPacket(RoutedPacket(relayPacket,
peerID, routed.relayAddress))
104          } else {
105          Log.d(TAG, "Relay
```

```

decision: NOT relaying packet type $
{packet.type}")
106      }
107    }

```

«وصلنا القرار الأخير يا سُكينة! لو ما
 نفع الطريق الدقيق، نقرّر: نصيح للكل أو
 لا؟»

السطر ١٠١: `val shouldRelay =`

`isRelayEnabled() &&`

`shouldRelayPacket(...)` — سؤالان لازم

الاثنان يكونان نعم (الرمز `&&` يعني "و"):

- `isRelayEnabled()` — هل الإعادة

مفعّلة أصلاً؟ (السطر ٢٤ اللي شرحناه!) -

`shouldRelayPacket(...)` — هل المنطق

يقول نعيد؟ (نشرحه بعد شوي، السطر
١٣١).

السطر ١٠٢-١٠٣: لو `shouldRelay` نعم،
نستدعي `relayPacket(...)` — يعني صح
للجيران! السطر ١٠٤-١٠٦: وإلا، نكتب
بالدفتر "قررنا: ما نعيدها"، وننتهي بهدوء.
السطر ١٠٧: القوس `}` يسكّر الدالة
الكبرى `handlePacketRelay`. خلصنا الباب
الرئيسي!

«يعني لازم شرطين: الإعادة مفعّلة، والمنطق
يوافق؟»

«بالضبط! الرمز `&&` يعني الاثنين سواء. لو
واحد لا، ما نعيد.»

الأسطر ١٠٩-١٢٩: الحارس الأول مكبراً (هل الرزمة إلي؟)

تذكرين الحارس الأول بالسطر ٤٧؟ هنا
تفاصيله الكاملة.

```
109      /**
110      * Check if a packet is
specifically addressed to us
111      */
112      internal fun
isPacketAddressedToMe(packet:
BitchatPacket): Boolean {
113          val recipientID =
packet.recipientID
114
115          // No recipient means
broadcast (not addressed to us specifically)
```



```

116         if (recipientID == null) {
117             return false
118         }

```

«الأسطر ١٠٩-١١١ لوحة تعريف: "تأكد

هل الرزمة موجهة إلنا بالذات." السطر

١١٢: internal fun

— isPacketAddressedToMe(...): Boolean

دالة جوابها نعم/لا (!is) و internal

يعني "تنشأف داخل التطبيق بس". السطر

١١٣: نطلع recipientID = هوية المستلم

المكتوبة بالرزمة. الأسطر ١١٥-١١٨: if

— (recipientID == null) return false

"لو ماكو مستلم محدد، يعني هذي رزمة

للـ (broadcast)، مو إلي بالذات" →

جواب: لا (false) «.

```

120          // Check if it's a broadcast
recipient

121          val broadcastRecipient =
delegate?.getBroadcastRecipient()

122          if (broadcastRecipient != null
&&
recipientID.contentEquals(broadcastRecipient
)) {

123              return false

124          }

125

126          // Check if recipient matches
our peer ID

127          val recipientIDString =
recipientID.toHexString()

128          return recipientIDString ==

```

myPeerID

129 }

«الأسطر ١٢٠-١٢٤: نسأل المندوب: "شنو

عنوان البثّ-

للكل؟" (getBroadcastRecipient) . ولو

المستلم يطابق عنوان البثّ، يعني للكل

برضو → جواب: لا (false) . الأسطر

١٢٦-١٢٨: آخر فحوص! نحول هويّة المستلم

لنصّ (toHexString) ، ونقارنها بهويّتي أنا

(myPeerID) . لو طابقت → الرزمة إلي!

جواب: نعم. السطر ١٢٩: القوس يسكّر

الدالة.

شفتِ يا سُكينة؟ ثلاثة أسئلة بالترتيب:

ماكو مستلم؟ للكل. عنوان بثّ؟ للكل.

يطابقني؟ إلي! منطق نظيف خطوة خطوة.»

«مثل ساعي البريد يشوف العنوان: لو ماكو اسم، يعلنه للكل؛ لو اسمي، يعطيني!»
«تشبيه جميل يا حبيبتى!»

الأسطر ١٣١-١٦٣: حكمة العاصفة (هل نعيد البث؟)

«هذا المقطع يا سُكينة هو حكمة تهدئة العاصفة بالكود الحقيقي! الدالة تقرّر: نعيد البث أو نسكت، حسب الزحام.»

```
131      /**
```

```
132      * Determine if we should relay  
this packet based on type and network  
conditions
```

```
133      */
```

```
134      private fun
```

```

shouldRelayPacket(packet: BitchatPacket,
fromPeerID: String): Boolean {

135          // Always relay if TTL is high
enough (indicates important message)

136          if (packet.ttl >= 4u) {

137              Log.d(TAG, "High TTL ($
{packet.ttl}), relaying")

138              return true

139          }

```

«الأسطر ١٣١-١٣٣ لوحة تعريف. السطر

١٣٤: الدالة `shouldRelayPacket` (هل

نعيد؟ — `style-is` جوابها نعم/لا).

الأسطر ١٣٥-١٣٩: أول قاعدة! `if`

(`packet.ttl >= 4u`) — "لو عدّاد القفزات

عالٍ (٤ أو أكثر)، أعد دائماً (`return`)

(`true`). "ليش؟ لأن العدّاد العالي معناه

الرسالة لسا بداية رحلتها ولازم تنتشر بقوة.
مثل صاروخ بأول إطلاقه – يحتاج دفعة
قوية.»

```
141          // Get network size for  
adaptive relay probability  
142          val networkSize =  
delegate?.getNetworkSize() ?: 1  
143  
144          // Small networks always relay  
to ensure connectivity  
145          if (networkSize <= 3) {  
146              Log.d(TAG, "Small network  
(${networkSize} peers), relaying")  
147              return true  
148          }
```

«السطر ١٤٢: نسأل المندوب: "كم عدد الجيران؟" (`getNetworkSize`). ولو ما عرف، اعتبره ١ (`1: ?`). الأسطر ١٤٤-١٤٨: قاعدة ثانية! `if (networkSize <= 3)` — "لو الجيران قليلين (٣ أو أقل)، أعد دائماً." ليش يا سُكينة؟»

فكرت سُكينة... «لأن لو قليلين، لو ما أعدت أنا، ممكن ما أحد يعيد، فتضيع الرسالة!»

«عبقريّة! بالضبط! بالقلّة، كل واحد مهم، فنعيد أكيد عشان ما تنقطع الشبكة.»

```
150          // Apply adaptive relay
probability based on network size

151          val relayProb = when {
152              networkSize <= 10 ->
```

```

1.0    // Always relay in small networks

153          networkSize <= 30 ->

0.85    // High probability for medium
networks

154          networkSize <= 50 ->

0.7    // Moderate probability

155          networkSize <= 100 ->

0.55    // Lower probability for large
networks

156          else ->

0.4          // Lowest probability
for very large networks

157      }

```

«وهنا قلب الحكمة يا سُكينة! الأسطر

١٥٧-١٥٠ تحسب احتمال الإعادة

حسب الزحام: (relayProb)

عدد الجيران	احتمال الإعادة	المعنى
حتى ١٠	١٠٠	أعد دائماً (قلّة)
حتى ٣٠	٠٠٨٥	غالباً
حتى ٥٠	٠٠٧	نصف-نصف تقريباً
حتى ١٠٠	٠٠٥٥	أحياناً
أكثر من ١٠٠	٠٠٤	نادراً (زحام شديد)

شوفي السحر: كل ما زاد الزحام، قلّ الاحتمال! بالزحام الشديد (أكثر من مئة جار)، أعيد بس ٤٠% من المرّات. ليش؟ لأن غيري بيعيدها! ما نحتاج كلنا نصيح. هذي بالضبط حكمة العاصفة اللي حكيها لك: جار واحد يكفي يغطّي المنطقة!»

«كل ما زاد الناس، أسكت أكثر، عشان
ما تصير ضجة!»

«بالضبط يا روح جدك! الكلمة when
معناها "حسب الحالة" — مثل سلسلة
"إذا... وإلا إذا..."»

```
159          val shouldRelay =  
Random.nextDouble() < relayProb  
  
160          Log.d(TAG, "Network size: $  
{networkSize}, Relay probability: $  
{relayProb}, Decision: ${shouldRelay}")  
  
161  
  
162          return shouldRelay  
  
163      }
```

«السطر ١٥٩: هنا نرمي الزهر!
يطلّع رقماً عشوائياً Random.nextDouble()»

بين ٠ و٠١. لو طلع أصغر من
 الاحتمال، نعيد. مثلاً لو الاحتمال ٠.٠٤،
 والزهر طلع ٠.٢ (أصغر) → نعيد. لو
 طلع ٠.٧ (أكبر) → نسكت. تذكرين جنبنا
 أداة Random بالسطر ٩؟ هنا نستعملها!
 السطر ١٦٠: نكتب القرار بالدفتر. السطر
 ١٦٢: نرجع القرار (shouldRelay). السطر
 ١٦٣: القوس يسكّر الدالة.»
 «يعني نرمي زهر، وحسب الزهر نعيد أو
 نسكت؟»

«بالضبط! العشوائية تخليّ بعض الجيران
 يعيدون وبعضهم يسكتون، بدون اتفاق مسبق.
 فما تصير عاصفة، وتوصل الرسالة. ذكاء
 جماعي بدون قائد!»

الأسطر ١٦٥-١٧١: الصياح الفعلي (relayPacket)

```
165      /**
166          * Actually broadcast the packet
for relay
167      */
168      private fun relayPacket(routed:
RoutedPacket) {
169          Log.d(TAG, "    Relaying packet
type ${routed.packet.type} with TTL $
{routed.packet.ttl}")
170
delegate?.broadcastPacket(routed)
171      }
```

«الأسطر ١٦٥-١٦٧ لوحة تعريف: "هنا
نبتّ الرزمة فعلياً." السطر ١٦٨: الدالة

relayPacket (لاحظي نفس اسم المتغيّر

بالسطر ٦٥، بس هذي دالة). السطر

١٦٩: نكتب بالدقتر "أعيد بثّ رزمة

نوعها كذا". السطر ١٧٠:

— delegate?.broadcastPacket(routed)

نطلب من المندوب: صح للجيران كلهم!

المدير نفسه ما يصيح؛ يطلب من

المتخصّص بالبثّ. السطر ١٧١: القوس

يسكّر.

شفتِ يا سُكينة؟ المدير يقرّر، والمندوب

ينفّذ. كل واحد شغلته.»

الأسطر ١٧٣-١٨٣: تقرير الحالة (getDebugInfo)

```
173      /**
174      * Get debug information
175      */
176      fun getDebugInfo(): String {
177          return buildString {
178              appendLine("=== Packet
179              Relay Manager Debug Info ===")
180              appendLine("Relay Scope
181              Active: ${relayScope.isActive}")
182              appendLine("My Peer ID: $
183              {myPeerID}")
184              appendLine("Network Size:
185              ${delegate?.getNetworkSize() ?: "unknown"}")
186          }
187      }
```

«الأسطر ١٧٣-١٨٣ دالة `getDebugInfo`

= "أعطني تقرير الحالة". مفيدة للمهندس

لمن يبي يعرف: المدير شغال؟ هويتي شنو؟

كم جار؟ `buildString` تبني نصاً سطرًا

سطراً (`appendLine`) = "أضف سطرًا".

مثل تقرير صغير يطبعه المدير عن نفسه.

ما تأثر على الشبكة — بس للفحص.»

«مثل لمن الطبيب يكتب تقريراً عن

حالتي!»

«تشبيهه حلوا! تقرير صحة المدير.»

الأسطر ١٨٥-١٩٢: الإطفاء بأدب (shutdown)

```
185      /**
```

```
186      * Shutdown the relay manager
```

```

187         */
188     fun shutdown() {
189         Log.d(TAG, "Shutting down
PacketRelayManager")
190         relayScope.cancel()
191     }
192 }

```

«الأسطر ١٨٥-١٩١ دالة shutdown =

"أطفئ المدير بأدب". لمن نبي نوقف كل شي: السطر ١٨٩: نكتب بالدفتـر "أطفئ

المدير". السطر ١٩٠: relayScope.cancel()

– نوقف ساحة العمال (تذكرين السطر

٣٤؟). نسرّح العمال بأدب عشان ما

يضلّون يشتغلون بالفراغ ويحرقون البطارية.

السطر ١٩٢: القوس الكبير } – يسكّر

بيت المدير كله! تذكّرين فتحناه بالسطر

١٧؟ هنا نسكّره. خلص المدير.»

«مثل ما نطقي الأنوار ونسكّر المحل آخر

اليوم!»

«بالضبط يا حبيبي. الإطفاء الأنيق جزء

من الأدب.»

الأسطر ١٩٤-٢٠٥: عقد المندوب (شنو لازم يعرف المندوب يسوّي؟)

```
194  /**
```

```
195  * Delegate interface for packet relay  
manager callbacks
```

```
196  */
```

```
197  interface PacketRelayManagerDelegate {
```

```
198      // Network information
```

```

199         fun getNetworkSize(): Int
200         fun getBroadcastRecipient():
ByteArray
201
202         // Packet operations
203         fun broadcastPacket(routed:
RoutedPacket)
204         fun sendToPeer(peerID: String,
routed: RoutedPacket): Boolean
205     }

```

«تذكرين المندوب يا سُكينة (السطر ٣١)؟
هنا عقده — قائمة بالخدمات التي لازم
المندوب يقدر يسوّيها. الكلمة `interface`

معناها "عقد/وعد" — مو تنفيذ، بس قائمة واجبات:

• السطر ١٩٩: `getNetworkSize(): Int`

— "أعطني عدد الجيران" (رقم صحيح).

• السطر ٢٠٠:

`getBroadcastRecipient(): ByteArray`

— "أعطني عنوان البث-للكل".

• السطر ٢٠٣:

`broadcastPacket(routed)` — "صح

بها الرزمة للكل".

• السطر ٢٠٤: `sendToPeer(peerID,`

`routed): Boolean` — "أرسلها لجار

محدد، وقل لي نجحت أو لا".

شفتِ يا سُكينة؟ المدير بالأسطر اللي فوق
استعمل هالخدمات الأربعة، وهنا يكتب
عقدها. أي مندوب يبي يخدم هالمدير،
لازم يقدر يسوي هالأربعة. مثل وصف
وظيفة: "مطلوب موظف يعرف: يعدّ الجيران،
يعطي عنوان البثّ، يصيح للكل، ويرسل
لواحد."»

«والمدير ما يهّمه كيف يسويها، بس إنه
يسويها؟»

«عبقريّة يا سُكينة! بالضبط — هذا سرّ
الترتيب: المدير يطلب الخدمة، والمندوب
يدبرها بطريقته. فنقدر نغيّر المندوب بدون
ما نلمس المدير.»

الأسطر ٢٠٧-٢١٧: الأداة الصغيرة (تحويل النصّ لبايتات)

```
207 private fun hexStringToPeerBytes(hex:
String): ByteArray {
208     val result = ByteArray(8)
209     var idx = 0
210     var out = 0
211     while (idx + 1 < hex.length && out
< 8) {
212         val b = hex.substring(idx, idx
+ 2).toIntOrNull(16)?.toByte() ?: 0
213         result[out++] = b
214         idx += 2
215     }
216     return result
217 }
```

«وآخر مقطع يا سُكينة! أداة صغيرة اسمها

`hexStringToPeerBytes` = "حوّل نصّ

الهويّة لبايتات". تذكرين السطر ٧٦ استعملها؟

هنا تفاصيلها:

• السطر ٢٠٧: اسم الأداة، تاخذ نصّاً

(`hex`) وترجع بايتات (٨ بايت).

• السطر ٢٠٨: `result = ByteArray(8)`

— نجهّز صندوقاً فاضياً من ٨

بايت.

• السطر ٢٠٩-٢١٠: `idx` و `out`

عدّادان نبدأهم من صفر (مؤشّر

القراءة، ومؤشّر الكتابة). `var` لأنهم

يتغيّرون كل دورة.

• السطر ٢١١: `while (...)` — كرّر

طول ما فيه حروف نقرأها ومكان
نكتب فيه.

• السطر ٢١٢: نأخذ حرفين من النصّ،

ونحوّلهم لرقم بايت (`toIntOrNull(16)`)

— ١٦ يعني "نظام ست عشري").

ولو فشل التحويل، حطّ صفر

(`0: ?`) — حماية من الفراغ.

• السطر ٢١٣: نخزّن البايت بالصندوق،

و `out++` يعني "خزّن ثم زد المؤشّر

واحد".

• السطر ٢١٤: `idx += 2` — تقدّم

حرفين بالنصّ (لأن كل بايت =

حرفين).

• السطر ٢١٦: `return result` —

رجّع الصندوق المملوء.

• السطر ٢١٧: القوس الأخير يسكّر

الأداة. وهنا... خلص الملف!

«حرفين يصيرون بايت واحد؟»

«إي يا حبيبي، النظام الست عشري كذا

— كل بايت يُكتب بحرفين. مثلاً `A1`

حرفان = بايت واحد. الأداة تترجم من

لغة الإنسان (نصّ) للغة الآلة (بايتات).»

خلصنا الملف كاملاً!

أغلق أبو حيدر الملف، ونظر لسُكينة

بفخر:

«يا روح جدّك، قرأنا ملفاً حقيقياً كاملاً،

سطراً سطراً، من ١ لين ٢١٧، بدون

ما نتخطّى أي سطر. وشفّتِ شنو فيه؟

• الأسطر ٩-١: جنبنا أدواتنا (استيراد).

• الأسطر ١١-١٦: كتبنا لماذا الملف

موجود.

• السطر ١٧: ولّدنا المدير وأعطيناه

هويّته.

• الأسطر ١٨-٣٤: جهّزنا مساعديه

(تنقيح، توقيع، مندوب، عمّال خلفية).

• الأسطر ٣٦-١٠٧: الباب الرئيسي —

حارسان، عدّاد القفزات، الطريق الذي،

القرار.

• الأسطر ١٠٩-١٢٩: كيف نعرف إن الرزمة إلنا.

• الأسطر ١٣١-١٦٣: حكمة العاصفة — احتمال الإعادة يقلّ مع الزحام.

• الأسطر ١٦٥-١٧١: الصباح الفعلي.

• الأسطر ١٧٣-١٩٢: تقرير الحالة والإطفاء الأنيق.

• الأسطر ١٩٤-٢٠٥: عقد المندوب.

• الأسطر ٢٠٧-٢١٧: أداة تحويل الهوية.

ملف واحد، شغلة واحدة (مين يعيد البث؟)، كل سطر له سبب. ما فيه سطر زائد، وما فيه سطر ناقص. هذا معنى الكود الجميل: مثل بيت مرتّب، كل شي بمكانه.»

نظرت سُكينة للشاشة المليانة أسطراً صارت
تفهمها كلها، وقالت:

«جدّو... قبل، كان الكود يخوّفني. الحين...
أشوفه قصة، كل سطر جملة، وكل دالة
فصل. وأنا أقدر أقرأها!»

ضمّها أبو حيدر وقال: «هذا أحلى ما
يصير يا حبيبي. لمن الكود يصير قصة
تفهمينها... وقتها تقدرين تكتبين قصتك
الخاصة. وتبنين شبكة... تجمع الأحبة.»

النهاية.

هذا كان ملف

PacketRelayManager.kt (٢١٧ سطراً)

من مشروع bitchat-android المفتوح

المصدر، مشروحاً سطرّاً سطرّاً. القصة
الكبرى (الفكرة والوحوش السبعة) بملف
القصة، والتفاصيل التقنية الشاملة بملف
الخطّة.